

CAKE: 面向前沿内核演进的编译器-智能体协同设计

Zihao Ye^{1*} Yingyi Huang^{1*} Hongyi Jin² Bohan Hou²
Junru Shao¹ Zhongming Yu¹ Jinqi Chen¹ Meghan Cowan¹
Shiyi Cao¹ Shanli Xing¹ Hanfeng Chen¹ Vinod Grover¹
Tianqi Chen^{1,2} Luis Ceze¹

¹NVIDIA ²Carnegie Mellon University

摘要

GPU 内核智能体与 GPU 编程语言的研究一直各自推进,而两者之间的鸿沟正是专家级内核难以复现之处。内核智能体通常把编译器当作固定的黑盒:它们改进候选生成、变异与排序,但环境只返回编译错误、正确性结果和端到端计时。这些信号既无法指出究竟是哪一项程序决策引发了同步失败、硬件契约违例或流水线停顿,也无法在前沿工作负载暴露能力缺口时随之扩展。与此同时,可供智能体编写的语言也并非为智能体而设计。块级 DSL 隐藏了区分专家内核与仅仅正确的内核所需的线程束专门化、屏障协同和存储层级放置;低级 DSL 虽暴露了这些控制,却要求掌握复杂的布局演算,使智能体更容易犯错且难以定位错误。本文提出 CAKE,对编译器与智能体进行协同设计。智能体编写 CAKE IR:一种带类型、显式体现硬件的调度表示。它无需布局代数即可提供细粒度控制,并携带足够信息,使验证器和代价模型能够在编译前推理程序;框架则以局部化的正确性与性能诊断回应。框架本身也是演进对象,因此反复出现的失败会转化为新的验证规则、IR 原语、代价模型校准和可复用优化策略,而非一次性的规避方案。

在 B200 上、实现细节隐藏且严格匹配的 Flash-KMeans 冷启动实验中(每种表示运行三次),在 8000 万 token 预算下,CAKE IR 最佳候选相对调优后的 FlashML 基线达到中位数 1.144 $\times$,而直接编写 CUDA/PTX 为 0.928 $\times$ 。在该冷启动基准之外,智能体生成的 Kimi Delta Attention 相对官方 FlashKDA 取得 2.05 $\times$ 的几何平均加速,并通过端到端服务验证。由调度器支持的 KNN 与 KMeans 系列在 400 多种形状上将性能提升 1.42 $\times$ -2.12 $\times$,另有四项内核改动以向上游 PR 的形式提供。CAKE 面向从 Ampere 到 Blackwell 的 NVIDIA GPU,并将单一形状的演进与库集成所需的泛化和分派阶段明确分开。

1 引言

编码智能体如今已能在自动测量正确性与性能的环境中编写和修订 GPU 程序 [1-7]。然而,多数系统仍把编程环境视作固定黑盒:智能体提出代码、编译、运行数值测试、测量延迟,再选择下一次修改。这个循环适用于局部调优,但程序崩溃时不会指出违反了哪项安全或硬件条件,单个延迟数字也无法解释究竟哪一项程序决策限制了性能。

专家内核程序员的工作方式不同。他们维持对工作负载的紧凑模型,围绕显式硬件资源推理,并把可复用规则迁移到不同内核之间。编译器其实已拥有将这一过程外显所需的大部分机制:结构化操作词汇、资源模型、合法性检查、静态分析、代价模型和 lowering 规则。CAKE 所追问的是,如何让智能体直接使用这些机制,以及当前沿工作负载暴露缺口时如何改进它们。

CAKE 作出三项承诺。第一,智能体编辑带类型的 IR,而不是原始 CUDA,因此代码生成之前就能检查硬件决策。第二,编译器返回局部化的正确性与性能诊断,而不是单一的通过/失败信号,从而让低成本分析在候选消耗 GPU 时间之前完成筛选。第三,框架本身也是演进对象:反复出现的失败会转化为验证规则、校准任务或新原语,并由语料测试把关。CAKE IR 通过智能体在生产内核语料上自底向上的抽象发现而设计,其准则是必须复现专家内核的物理调度与性能(图 2;过程详见原文附录)。该框架同样主要由智能体维护,并由人工合并门控把关。

系统支持两种入口,对应内核工作的两种实际来源。它既可以从 FlashInfer 或 CUTLASS 等库中的生产内核出发并继续演进该实现;也可以在没有成熟参考实现的工作负载上,从高层描述或 Triton 实现出发,让智能体在 CAKE IR 中选择线程束专门化、布局和流水线结构,由编译器验证程序并生成 CUDA。

本文同时报告上述第二种模式和第一种模式下的结果。CAKE 面向从 Ampere 到 Blackwell 的 NVIDIA GPU,其已验证语料覆盖数十种内核族,包括注意力与线性注意力、稠密/分组/量化 GEMM、MoE 数据流、归一化、量化、Top-K、KNN、KMeans 和融合图内核。第 5 节评估针对调优基线的重复冷启动演进,报告 Kimi Delta Attention [8]、Gated DeltaNet [9] 和稀疏注意力等模型架构上的前沿内核合成,并报告对既有内核族的复现。第 6 节进一步处理基准数字通常略过的一步:如何把针对单一形状调优的内核转化为服务库可在任意形状下调用、由调度器支持的内核族。

*同等贡献。

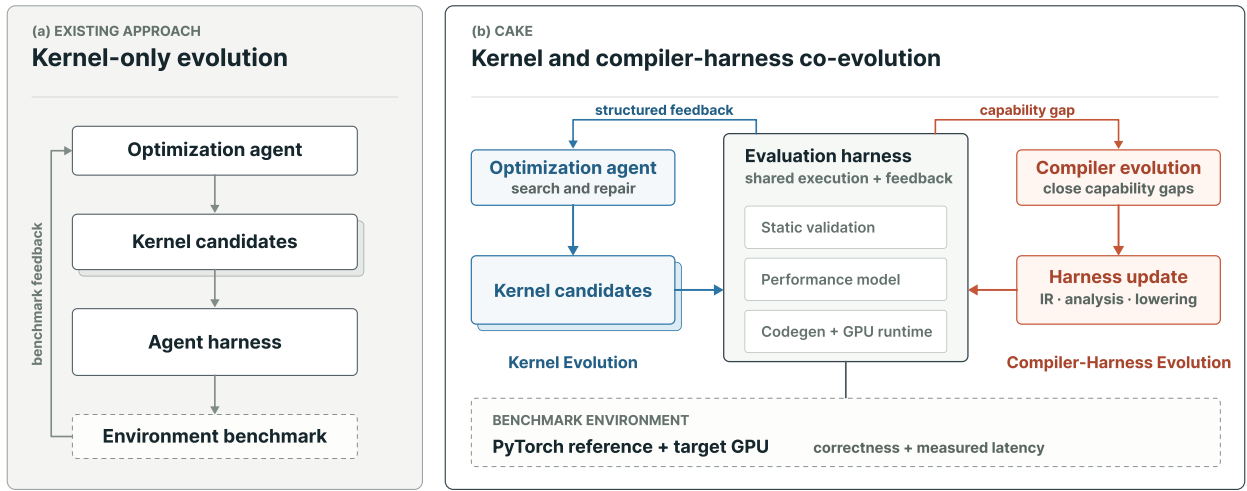


图 1: CAKE 概览。内核演进以结构化编译器证据为输入；编译器演进构成外层循环。

2 程序表示

CAKE 将 CAKE IR 作为面向智能体的程序表示，并生成 CUDA/PTX 以执行程序。结构化分析和性能建模在 CAKE IR 上运行，常规的 sanitizer 与 profiler 反馈则来自生成的代码。

2.1 自底向上的 IR 演进

CAKE 并非从预定义的 CAKE IR 词汇表起步。它的初始材料是生产内核语料与硬件设计原则。智能体从这些内核中识别反复出现的调度模式或缺失能力，修订 IR 及其编译器支持，再移植内核并在修订后的系统上验证。下一个内核族，或验证暴露出的缺口，会再次启动同一循环。这个循环既产生了当前的 IR，也持续扩展它（图 2；详细过程见原文附录）。

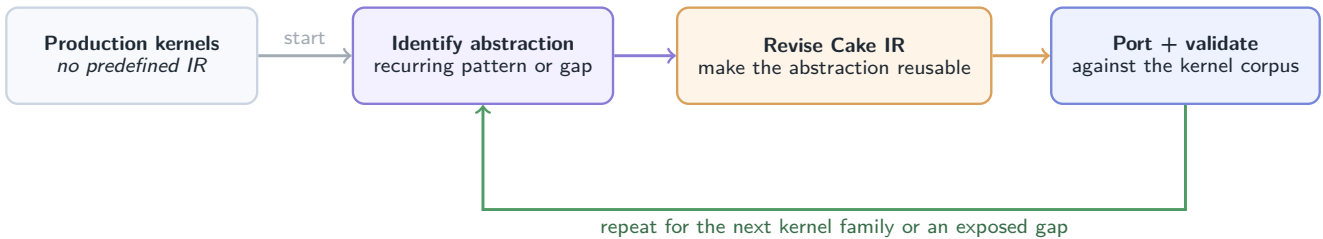


图 2: CAKE IR 从内核而不是固定语言设计中演进。初始语料只进入一次；此后三步循环会针对每个新内核族或能力缺口重复执行。

2.2 Cake IR 中的显式机器调度

CAKE IR 记录应当如何驱动机器：哪些线程束承担哪些角色、哪些缓冲区以多深的阶段数进行暂存、哪个屏障控制哪一次交接、哪种指令形式消费哪一个操作数。相应的分工是：调度声明要做什么，lowering 推导如何完成。屏障地址、相位位、TMEM 偏移、描述符编码和线程束身份都由声明计算得到，而不是由智能体手写。

一个程序组合了显式操作、声明式资源、线程束角色以及网格与流水线配置（图 3）。四项性质发挥核心作用。类型检查词汇：计算、数据移动、同步、数学和线程束控制采用固定 IR 词汇，而非嵌入式 C 或 PTX。声明式资源：内存区域、同步状态和流水线只声明一次，因此 IR 掌握每个缓冲区的形状、数据类型和生命周期。显式角色：线程束组均有名称，跨角色交接全部可见，而不是隐含约定。自动推导元数据：这些声明的机械性结果由 lowering 生成，无需人工编写。

其收益在于，分析可以在代码生成之前依据显式调度决策进行推理。因此，框架能够把发现绑定到受影响的资源、角色或阶段，而不是只返回后端错误或程序挂起。语言目录、资源模型和设计原则详见原文附录。

布局。 CAKE 有意不把布局设为一等抽象。CAKE IR 不要求智能体操纵布局代数，而是直接在调度中记录存储与访问决策。编译器随后检查生产者与消费者表示是否兼容目标硬件（详见原文附录）。

```

1 @cake.schedule()
2 def fmha_fwd(lm, Q: LM.tma3d, O: LM.tma2d, seqlen_q: LM.i32):
3     # declare resources: named resources, not raw addresses
4     pool = lm.smem(98304)
5     smem_q = pool.view(offset=0, shape=(128,128), dtype=lm.bf16, stage=3)
6     tmem_acc = lm.tmem(cols=0, width=128, shape=(128,128), dtype=lm.f32)
7
8     # declare roles: warp groups with assigned work
9     load = lm.role(warps=[0])
10    mma = lm.role(warps=[1])
11    pipe = lm.pipeline(stages=3)
12    q_full = lm.barrier(count=3, prod=[load], cons=[mma],
13                        init_count=1, pipeline=pipe)
14
15    with load:
16        for stage in lm.range(0, 3):
17            smem_q.tma_load(Q, coords=(0,0,stage), stage=stage, barrier=q_full)
18
19    with mma:
20        for stage in lm.range(0, 3):
21            lm.wait(q_full, stage=stage)
22            lm.fence_proxy()
23            lm.mma(tmem_acc, smem_q[stage], smem_q[stage], init=(stage == 0))

```

图 3: 一个 CAKE IR 调度片段。资源与角色通过声明给出；生产者-消费者交接是显式屏障；地址、相位与线程束身份由 lowering 推导。

2.3 体系结构与 lowering

同一套调度语言面向从 Ampere 到 Blackwell 的 NVIDIA GPU，因此“角色-屏障-流水线”调度在结构上可移植，而指令准入与 lowering 仍因目标而异。CAKE 会精确映射所连接的 GPU；对于不支持的目标，它会明确报告，而不会静默替换为其他体系结构；只有目标专用校准可用时才会输出性能估计。详细设备矩阵和后端说明见原文附录。

3 编译器框架

该框架是围绕 CAKE IR 构建、面向智能体的环境。人类给出预期分析的高层描述，智能体则在验证约束下实现、维护和细化这些分析。在内核演进期间，低成本分析会在候选进入昂贵的 GPU 运行之前完成排序与筛选。

3.1 分析与验证

程序安全与硬件一致性。 编译之前，框架会检查带类型调度中大类的同步、内存安全、数据流、资源、指令和数据表示违例。这些检查会拒绝许多在数学上看似合理、却不符合目标执行模型的候选。每项发现都指出受影响的程序区域与被违反的契约类别，从而通过稳定的分析接口为智能体提供可操作的修复目标。

数值正确性。 对于每个工作负载，我们在不同形状和输入分布上比较内核输出与参考输出。最终验收要求在相应目标框架中进行端到端评估。

性能建模。 经校准的代价模型估计候选性能，并返回高层瓶颈归因和优化指导。模型用于候选排序与筛选；设备实测和性能分析仍是最终事实依据。

表 1 按外部可见功能而非单个 pass 或具体实现汇总该套件。关键接口是其契约：阻断式检查以局部化理由拒绝候选，报告描述可能的性能限制，提示则给出非阻断式优化建议。

表 1: CAKE 框架向外暴露的分析与验证类别。类别描述用户可见行为，而非内部 pass。

类别	处置方式	目的
程序安全	编译前门控	识别同步、顺序和内存使用风险
硬件一致性	编译前门控	强制执行受支持的资源、指令和体系结构契约
数据一致性	编译前门控	检查数据流及生产者-消费者表示兼容性
调度语义	编译前门控	检查声明式调度的结构不变量
数值验证	执行门控	将编译后输出与权威外部参考进行比较
性能分析	报告	估计代价并识别大类瓶颈
优化指导	提示	在不阻断编译的情况下建议有潜力的修订

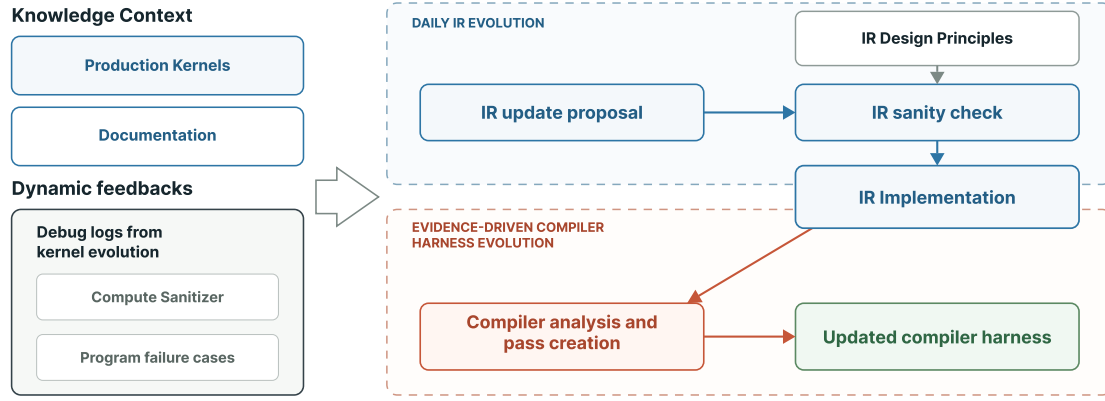


图 4: 证据驱动的编译器演进。语料与运行时证据共同驱动经过验证的编译器改动。

3.2 编译器演进

CAKE 让编译器与内核共同演进。内核候选、验证结果、基准测试和失败报告共同构成提出并验证编译器改动的证据。

编译器演进沿两条互补路径进行(图 4)。第一条路径中,智能体检查生产内核与硬件文档,寻找缺失的 Blackwell 模式,例如新指令形式、资源类型、描述符变体和同步惯用法,并据此提出编译器改动。每项提案在实现前都会依据 CAKE IR 的设计原则进行检查,其中包括性能透明性和验证友好性。第二条路径中,智能体利用失败候选的反馈,例如 sanitizer 报告、失败用例、正确性不匹配和调试日志,将反复出现或代价高昂的失败模式提炼为新分析:不透明的运行时崩溃会变成验证器规则,反复出现的非法 lowering 模式会变成静态检查,系统性误判则会变成校准目标。

两条路径相互耦合。新原语向编译器暴露更多硬件事实,使更强的分析成为可能;新分析反过来约束未来原语的设计空间。编译器改动必须通过整个内核语料的测试门控,因为原语及其分析必须一同演进:只有语法而缺乏效果与合法性规则会降低 IR 的可分析性,而未经语料验证的新验证器规则则可能拒绝合法内核。

4 智能体 workflow

外部工作负载契约是稳定的权威依据。一次运行包含四个阶段:生成结构不同的 CAKE IR 候选;在消耗 GPU 时间前,利用 IR 构造检查、验证器硬门控和代价模型排序筛选候选;针对外部预言机评估幸存候选,并收集基准测试与 profiler 证据;最后根据诊断,将所得证据路由到候选、验证器、代价模型或 IR 词汇。工作负载契约固定形状、预言机、容差、硬件和允许访问的参考,而保留的结果使决策可审计,也使反复出现的发现可复用。

所有报告的智能体任务均使用 GPT-5.6-sol [10],推理强度为 xhigh。固定模型与智能体脚手架,使第 5 节的比较可归因于环境,而不是模型能力差异。

5 评测

评测回答三个问题:编译器框架能否推动重复冷启动演进超过调优后的基线;CAKE 能否在没有低级实现参考的情况下合成前沿内核;以及它能否对照最先进基线复现专家内核。

实验协议。 所有测量均在 B200 上使用 GPU 端正确性检查与 CUPTI 计时,并在每个计时样本前清空 L2 缓存。每个报告的候选都会在列出的形状上编译、检查正确性并运行基准测试。

重复冷启动实验固定编码智能体及其脚手架、模型和推理强度、任务描述、正确性预言机、基准测试框架以及单一目标形状。Flash-KMeans 实验使用隔离的 B200 冷启动环境;该环境提供任务规范、正确性预言机和基准接口,但隐藏目标的低级实现。处理组编写带类型的 CAKE IR,对照组则直接编写 CUDA C++ 和内联 PTX。我们报告供应商 token 消耗量与主动演进时间。表 2 汇总了每个实验组在 8000 万 token 预算下的三次匹配运行,图 5 展示其符合条件的性能检查点。汇总项采用中位数 [min, max];详细停止条件与计时核算保存在配套制品中。

参考访问策略。 允许访问哪些参考取决于所回答的问题。在冷启动与前沿内核合成中,智能体可以检查数学规范、评测契约、正确性预言机和高层代码,但不得检查 CUDA、PTX、SASS 或同等生成源码等低级目标实现。这些参考已编码了实验要求智能体自行发现的调度决策。外部实现仍可作为黑盒性能基线由基准框架执行,但其内部细节对智能体不可见。在已知内核复现中,智能体可以查看参考实现。Flash-KMeans 的限制在隔离冷启动环境

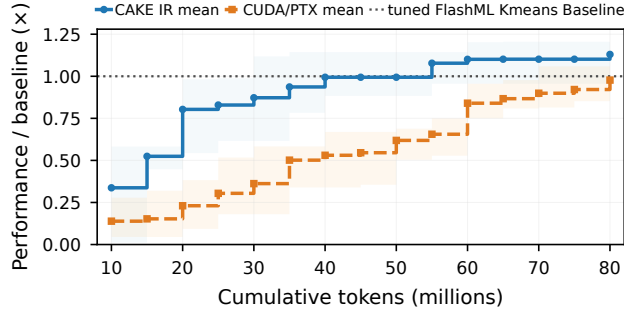


图 5: B200 上 Flash-KMeans 固定形状冷启动的达成情况。从 10M 到 80M，每隔 5M token 的预算点，每次运行都贡献截至该点最佳的已验证加速比。曲线表示三次运行均值，阴影带表示运行间最小值-最大值，水平线表示调优后的 FlashML K-means 基线。

中执行，并在事后审计。直接 CUDA/PTX 组改变的是所编写的表示，而不是参考策略：该组可以编写低级代码，但不能检查已有的目标实现。

Flash-KMeans 冷启动工作负载。 为测试从实现隐藏的冷启动出发能否复现结果，我们使用 Flash-KMeans [11]。这是一个精确 k -means 工作负载，动机来自 Sparse VideoGen2 中的语义感知 token 置换 [12]。一次 Lloyd 迭代主要由两个 BF16 内核主导，二者占端到端时间的 95% 以上：assign 计算每个 token 到全部 K 个质心的平方欧氏距离并返回 arg-min；centroid_update 则把每个簇内的 token 归约为逐簇求和与计数。两者具有不同特征：assign 是计算受限的 BF16 GEMM 加归约内核，而 centroid_update 对带宽和原子操作争用敏感。我们聚焦 assign；其在 $B=32$ 、 $N=65,536$ 、 $K=1024$ 、 $D=128$ 下使用 BF16 输入与 FP32 累加器，覆盖 Tensor Core 流水线和标量尾处理。所有运行采用相同模型、预言机与基准测试。性能归一化到经过调优的 FlashML KMeans Triton 实现，该基线实测为 0.938 ms。

表 2: B200 上匹配的三次 Flash-KMeans 冷启动比较。

表示	截至 80M 达到平台	主动演进 时间 (小时)	80M 时 最佳性能
CAKE IR	3/3	1.89 [1.02,2.33]	1.144 [1.041,1.205]
直接 CUDA/PTX	0/3	3.73 [3.59,4.34]	0.928 [0.852,1.151]

演进轨迹。 图 5 展示性能增益何时出现，从而补充表 2 中的终点汇总。CAKE IR 组的均值在 5500 万 token 时越过调优后的 FlashML 基线，并继续改善；直接 CUDA/PTX 组的均值到 8000 万 token 截止点仍低于基线。

在匹配运行中，CAKE IR 在 8000 万 token 前有 3/3 次达到预先规定的平台标准，而直接 CUDA/PTX 为 0/3。相对调优后的 FlashML 基线，前者的最佳达成中位数为 1.144 $\times$ ，后者为 0.928 $\times$ ；主动演进时间中位数分别为 1.89 小时与 3.73 小时。

5.1 前沿内核合成

这里所说的“前沿”具有操作性定义：智能体必须在不检查低级目标实现的情况下发现物理调度。已有实现仍可作为黑盒评测基线。协同演进的 IR 最有可能在这一模式中发挥作用，因为搜索无法锚定到已知优良设计；与此同时，这一模式也最能暴露框架的不足，因为能力缺失会直接表现为智能体完全无法表达某种调度。

新兴模型架构。 Kimi Delta Attention (KDA) 是最清晰的案例。官方 FlashKDA 仅作为黑盒计时基线使用，其源码与生成代码均不提供给智能体。一个兼容 FlashKDA 的 prefill 实现覆盖固定输入、打包变长输入和尾部输入，在六个 B200 BF16 形状上相对该基线达到 2.05 $\times$ 的几何平均加速。它在验证契约上逐位正确，并已在 SGLang 下的端到端 Kimi-K3 服务中得到验证。生成的 CUDA 可见于 FlashInfer PR #4262¹，因此下游用户无需依赖 CAKE。独立的 decode 路径在 30 个公共 API 形状上相对上游 FlashInfer 达到 1.14 $\times$ 的几何平均性能 (FlashInfer PR #4279²)。不同于带尾处理的 GEMM，KDA 包含必须跨块保持存活的循环状态，因而能有效检验调度表示。相应的两阶段源会话轨迹见原文附录。

¹github.com/flashinfer-ai/flashinfer/pull/4262

²github.com/flashinfer-ai/flashinfer/pull/4279

对比 FlashInfer, Gated DeltaNet 的 prefill 与 speculative-decode 路径在保留模型循环状态的同时提升了性能。MiniMax 稀疏注意力进一步表明,同一表示能够支持 prefill 与 decode 两种路径上的稀疏注意力族。这些都是分派族而不是单一内核:不同物理调度以独立 CAKE IR 程序的形式保留在同一逻辑入口之后。

参考引导的生产演进。 TinyGEMM 为上述冷启动前沿实验提供了互补的生产案例。智能体从 FlashInfer 中源自 TensorRT-LLM 的小 M BF16 内核出发,生成了浅层和深层流水线组成的自适应内核族,其中包括 PDL 变体和小于 8 的批大小。FlashInfer PR #4274³ 报告在 35 个标准形状以及更广泛回归测试上,内核时间的几何平均下降 18-23%。在 B200 与 GB300 上,GPT-OSS-20B 和 GPT-OSS-120B 的贪心解码保持逐位一致;另一次独立的 SGLang GPT-OSS-120B 实验报告,在 TP1、并发度 128 时输出吞吐量最高提升 7.6%,而 TP4 上的差异处于测量噪声范围内。先前的四形状搜索与针对小形状的后续优化详见原文附录。

通信密集型巨型内核演进。 Alpha-MoE 的原始实现 [13] 面向 Hopper。CAKE 智能体从该实现出发,成功将其 W8A8 融合 MoE 巨型内核重写到 Blackwell。所得实现检验演进中的编译器能否表达围绕 Tensor Core 计算的数据交换,而不只是计算本身。它把路由 gather、两次投影、激活、重新量化和按路由权重累加输出融合到一个设备程序中。

相对于 FlashInfer 中源自 TensorRT-LLM 的预路由 API,它在 $N=256$ 与 $N=512$ 时的 API 级端到端加速比分别为 $6.204\times$ 和 $4.025\times$ 。以 GPU span 重新测量后,加速比分别为 $1.215\times$ 与 $1.170\times$,从而单独体现有效 GPU 执行上的改进。更大的 API 级增益还来自启动/调度融合所减少的调度空隙,以及更简单的工作空间处理:参考实现启动五项 GPU 活动,而 Alpha-MoE 只使用一次输出重置与一个巨型内核。对应的 FlashInfer 贡献是 FlashInfer PR #4287⁴。归一化重写轨迹见原文附录;其分母是首个正确的 CAKE 检查点,而不是 TensorRT-LLM。

5.2 已知内核复现

为了验证输出是否达到生产质量,我们还针对现代 LLM 执行路径上具有最先进基线的已知算子族进行实验:注意力前向/反向与 decode、低精度 GEMM,以及 MQA/MLA logits 与 decode 内核。参考实现来自 TensorRT-LLM [14]、CUTLASS [15]、DeepGEMM [16]、FlashAttention-4 [17] 和 FlashInfer [18]。这些案例检验的问题不同于前沿内核合成:已有专家结构时,协同演进的框架能否帮助智能体在保持正确性的同时追平或超过高度优化的内核?比较设置本身是固定的,包括内核集合、每种变体列出的参考以及评测形状,因此重跑只会改变数值,不会改变待检验的主张。完整的逐内核矩阵、测量形状、相对性能和经审计的实现规模均见原文附录。

在当前测量的 11 组固定比较中,10 组达到或超过所列参考,剩余一组达到参考性能的 96.5%。最强结果是两个 MQA indexer,均约为 $1.27\times$ 。所有比较都通过各自内核特定的正确性门控,并采用 CUPTI GPU span 的中位数。

理解这些结果时需考虑两个因素。低于参考的变体通常反映编译器集成成熟度,而不是采用了不同算法目标:当内核所需特性仍在集成到编译器和代码生成器中时,提交的制品采用当前最接近的受支持策略。相反,表现最强的 indexer 并非对参考内核的忠实转写。在移植过程中,智能体探索了原实现中没有的优化,并保留通过正确性与基准测试的变体,因此高于参考的结果不仅来自转写保真度,也来自搜索。

代码行数列仅作描述,不能作为跨语言的生产力或可读性指标。表中每个 CAKE IR 实现都短于经审计的参考设备核心,但语言和统计范围不同。该比较只能说明,受测硬件调度可以在 CAKE IR 中得到紧凑表示,同时保留分析与 lowering 所需的决策。

5.3 内核组合

前述评测没有充分反映该框架目前所支撑的广度。经过验证的语料包含 400 多个静态与编译用例,以及横跨约 28 个内核族的 399 个 GPU 正确性用例,覆盖注意力、稠密与稀疏 GEMM、MoE、量化、归一化、状态空间模型、KNN 和 KMeans。它还包含从 Ampere 到 Blackwell 的体系结构专用路径。

除第 5.1 节的前沿内核之外,这套语料的另一突出性质是组合能力。由于角色、屏障与缓冲区都通过声明而不是隐含约定给出,通常需要拆成多个内核的调度可以表达为一个设备程序。BatchAttention 组合 decode 与 prefill 工作,Alpha-MoE 和 mega-MoE 内核族则在物化中间结果的情况下融合路由、专家计算和输出累加。语料还包含 100 多个 TensorRT-LLM 移植,覆盖注意力、MLA decode 与 MoE 内核。原本面向 Hopper 的 Alpha-MoE W8A8 内核由 CAKE 智能体重写到 Blackwell,说明即使目标指令发生变化,调度结构仍可保留。

四项上游改动分别涵盖 KDA prefill、KDA decode、TinyGEMM2 和 Alpha-MoE。

³github.com/flashinfer-ai/flashinfer/pull/4274

⁴github.com/flashinfer-ai/flashinfer/pull/4287

6 从单一调优形状到程序库

到目前为止，所有工作都在优化一个形状，而程序库必须接受调用者传入的任意形状。弥合这一鸿沟并不是简单地在更多形状上运行内层循环；它是一个具有不同目标、排序信号和失败模式的独立阶段，CAKE 也据此将其单独处理。

分离目标。 精确形状为内层循环提供清晰分母，并允许激进专门化。如果以广泛覆盖率为内层循环评分，会削弱这一信号。因此，泛化只会在已有强力逐形状种子之后开始，并在固定工作负载上按照包含调度器的性能评分。错误或缓慢的种子会返回内层循环，而不会被隐藏在路由之后。

构建与验证组合。 泛化阶段把已测种子按形状桶分组，生成专门化或共享变体，并在显式回退路径之前排列其守卫条件。调优可以改变实现参数，但不能改变输入形状。在报告聚合结果之前，验证会覆盖代表性和留出输入、边界与尾部用例、重叠或缺失的守卫条件，以及回退路径。

防止评测泄漏。 有效形状域在调优前声明。调度器谓词可以划分该域，但不得引入方便的新评测行。覆盖范围通过同一起来源中确定性的未见分片扩展。这样的分离可以避免在用于宣称泛化性能的集合上调优调度器。

在语料中的具体形态。 第 5.1 节中的若干内核已经是组合而非单一调度。KNN build 使用粗粒度外层内核族，并在内部按形状专门路由；KMeans 则分派到数量更少的最终路由桶。注意力组合也在同一逻辑入口后路由不同的 decode 和 prefill 调度。逐路由分解见原文附录。

经典机器学习工作负载最清楚地体现了该阶段的收益，因为这些组合足够庞大，单一形状无法左右结果。在 GB200 上，贡献到 FlashLib 的泛化 KNN build、KNN search 与 KMeans 实现取得包含调度器的结果。我们报告 G_{span} ，即逐形状加速比的无权几何平均；每个加速比定义为参考实现的 CUPTI GPU span 中位数除以 CAKE 的 CUPTI GPU span 中位数。三者在 112、198 和 124 个形状上的 G_{span} 分别为 $1.418\times$ 、 $2.116\times$ 和 $1.803\times$ ，没有错误输出，KNN 的 recall 为 1.0。这些完整组合结果回答的问题不同于表 2 中三次运行、单一形状的 Flash-KMeans 冷启动群组：前者在分派后评估完整形状集合，后者则演进一个精确形状。由于两组测量使用不同主机、形状分布、基线和协议，它们之间的差异本身不能视为对泛化成本的测量。

一项策略限制了泛化的实际成本：该阶段会让单一物理调度覆盖尽可能大的形状域，只有当该域确实需要实质性的调度改变时才引入新调度。路由复杂度必须由实测工作负载收益证明合理。由于每条路由都是独立的 CAKE IR 程序，各种备选方案仍可分别分析与测试。

7 相关工作

GPU 编程系统。 现有 DSL 分成两类，而两类都不便于智能体驱动的内核开发。高层与中层的块级 DSL，例如 Triton [19]、Helion [20]、TileLang [21] 和 cuTile [22]，通过块抽象和通常自动化的调度隐藏硬件；但这种不透明性阻碍智能体表达线程束专门化、屏障协同与存储层级放置，而这些正是专家内核区别于仅仅正确的内核之处。CuTe DSL [23] 等低级 DSL 暴露硬件控制，却要求布局代数等领域专长，学习负担很重，而且布局选择错误时会产生脆弱代码。Gluon [24] 位于两者之间：它复用 Triton 编译器栈，同时暴露对布局、数据移动和异步操作的更低级控制。CAKE 通过与智能体共同设计 IR 来填补这一缺口：CAKE IR 无需布局演算即可提供细粒度硬件控制，并且会持续演进；智能体发现无法表达的模式时加入新原语，遇到新的错误类别时细化 pass，预测失败时重新校准代价模型。

编译器分析与调度。 TVM [25]、XLA [26]、MLIR [27]、TensorIR [28]、Ansor [29] 和 MetaSchedule [30] 为程序赋予适合分析与优化的结构。Graphene [31, 32]、Twill [33] 和 Tawa [34] 对异步 GPU 执行、流水线或线程束专门化建模。CAKE 认同结构化程序能够支持有用分析这一原则，但它把编译器发现置于智能体演进循环中，并让反复出现的内核证据推动框架演进。

内核智能体。 KernelBench [1] 建立了将高层算子翻译为高效内核的通用评测场景。若干系统使用人工设计的循环，让 LLM 根据编译、正确性或 profiler 反馈修订内核 [4–6]。KernelBlaster [3] 增加了可持久化、可检索的 CUDA 知识库，从以往优化经验中持续填充内容。KernelEvolve [2] 与 EvoEngineer [35] 支持对内核候选进行演化搜索。AVO [36] 用自主编码智能体充当演化变体算子，替代固定的变异和交叉启发式。K-Search [37] 把规划与实现分离，并使用 LLM 世界模型引导搜索。AutoTriton [38] 通过监督微调 and 强化学习训练 Triton 模型。CUDA Agent [7] 扩展了面向 CUDA 生成与优化的智能体强化学习。这些方法演进搜索过程、积累的记忆或模型权重，但保留选定的 DSL 与评测环境。CAKE 针对的是与之互补的层次：它改变被搜索的表示，以及返回给智能体的结构化编译器证据。

演进系统。 FunSearch 与 AlphaEvolve [39, 40] 使用基于种群的演化搜索, 并由 LLM 生成程序变异。Automated Design of Agentic Systems [41] 使用元智能体和既往发现档案, 迭代提出以代码表示的智能体实现。Darwin Gödel Machine [42] 迭代修改编码智能体实现, 对每个变体进行经验评估, 并将变体保留在开放式档案中。Meta-Harness [43] 优化固定 LLM 周围的代码。更广义的自定义系统议程 [44] 则研究能够改变自身机制与抽象的 AI 运维系统。CAKE 的区别在于被演进的对象: 编码智能体与基础模型保持固定, 反复出现的内核证据则在语料测试和预定义合并门控下, 推动领域专用编译器框架中的 IR 词汇、分析和代价校准发生变化。

8 讨论与结论

CAKE 目前面向从 Ampere 到 Blackwell 的 NVIDIA 体系结构。调度语言、角色模型和分析基础可以跨这些目标迁移, 而指令形式、合法性规则与代价锚点仍然依赖具体体系结构; 这才是衡量迁移的真实成本。对于非 NVIDIA 目标, 这一成本尚未测量, 因为其后端 lowering 路径也必须重建。现有覆盖并不均衡: 多数性能证据来自 B200, 计时模型仅针对 B200 与 H100 校准, 并会拒绝在其他目标上作出预测。静态分析与性能模型仍有意保持不完备; 它们在演进期间用于排序与筛选, GPU 执行始终是真实依据 (详见原文附录)。编译器演进在合并门控处仍由人类引导。

CAKE 把编译器环境视为内核智能体的演进型协作者。CAKE IR 将硬件决策暴露给结构化分析, 编译器演进循环则把反复出现的失败转化为可复用的编译器知识。我们希望这项工作能推动编译器演进与编译器-智能体协同设计的进一步研究。

参考文献

- [1] Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. Kernelbench: Can llms write efficient gpu kernels?, 2025. URL <https://arxiv.org/abs/2502.10517>.
- [2] Gang Liao, Hongsen Qin, Ying Wang, Alicia Golden, Michael Kuchnik, Yavuz Yetim, Jia Jiunn Ang, Chunli Fu, Yihan He, Samuel Hsia, Zewei Jiang, Dianshi Li, Uladzimir Pashkevich, Varna Puvvada, Feng Shi, Matt Steiner, Ruichao Xiao, Nathan Yan, Xiayu Yu, Zhou Fang, Roman Levenstein, Kunming Ho, Haishan Zhu, Alec Hammond, Richard Li, Ajit Mathews, Kaustubh Gondkar, Abdul Zainul-Abedin, Ketan Singh, Hongtao Yu, Wenyuan Chi, Barney Huang, Sean Zhang, Noah Weller, Zach Marine, Wyatt Cook, Carole-Jean Wu, and Gaoxiang Liu. Kernelevolve: Scaling agentic kernel coding for heterogeneous ai accelerators at meta, 2026. URL <https://arxiv.org/abs/2512.23236>.
- [3] Kris Shengjun Dong, Sahil Modi, Dima Nikiforov, Sana Damani, Edward Lin, Siva Kumar Sastry Hari, and Christos Kozyrakis. KernelBlaster: Continual cross-task CUDA optimization via memory-augmented in-context reinforcement learning. *arXiv preprint arXiv:2602.14293*, 2026. URL <https://arxiv.org/abs/2602.14293>.
- [4] Genghan Zhang, Shaowei Zhu, Anjiang Wei, Zhenyu Song, Allen Nie, Zhen Jia, Nandita Vijaykumar, Yida Wang, and Kunle Olukotun. Accelopt: A self-improving llm agentic system for ai accelerator kernel optimization, 2026. URL <https://arxiv.org/abs/2511.15915>.
- [5] Charles Hong, Sahil Bhatia, Alvin Cheung, and Yakun Sophia Shao. Autocomp: A powerful and portable code optimizer for tensor accelerators, 2025. URL <https://arxiv.org/abs/2505.18574>.
- [6] Kaiming Cheng, Laura Wang, Jack Khuu, Mark Saroufim, Wenyuan Chi, Jiannan Wang, and Joe Isaacson. KernelAgent: Hardware-guided GPU kernel optimization via multi-agent orchestration. PyTorch Blog, 2026. URL <https://pytorch.org/blog/kernelagent-hardware-guided-gpu-kernel-optimization-via-multi-agent-orchestration/>.
- [7] Weinan Dai, Hanlin Wu, Qiying Yu, Huan ang Gao, Jiahao Li, Chengquan Jiang, Weiqiang Lou, Yufan Song, Hongli Yu, Jiase Chen, Wei-Ying Ma, Ya-Qin Zhang, Jingjing Liu, Mingxuan Wang, Xin Liu, and Hao Zhou. Cuda agent: Large-scale agentic rl for high-performance cuda kernel generation, 2026. URL <https://arxiv.org/abs/2602.24286>.
- [8] Kimi Team. Kimi linear: An expressive, efficient attention architecture, 2025. URL <https://arxiv.org/abs/2510.26692>.
- [9] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving mamba2 with delta rule. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=r8H7xhYPwz>.
- [10] OpenAI. GPT-5.6 Sol Model. OpenAI API Documentation, 2026. URL <https://developers.openai.com/api/docs/models/gpt-5.6-sol>. Accessed: 2026-08-09.
- [11] Shuo Yang, Haocheng Xi, Yilong Zhao, Muyang Li, Xiaozhe Fan, Jintao Zhang, Han Cai, Yujun Lin, Xiuyu Li, Kurt Keutzer, Song Han, Chenfeng Xu, and Ion Stoica. Flash-kmeans: Fast and memory-efficient exact k-means. *arXiv preprint arXiv:2603.09229*, 2026. doi: 10.48550/arXiv.2603.09229. URL <https://arxiv.org/abs/2603.09229>.

- [12] Shuo Yang, Haocheng Xi, Yilong Zhao, Muyang Li, Jintao Zhang, Han Cai, Yujun Lin, Xiuyu Li, Chenfeng Xu, Jianfei Chen, Song Han, Kurt Keutzer, and Ion Stoica. Sparse videogen2: Accelerate video generation with sparse attention via semantic-aware permutation. *arXiv preprint arXiv:2505.18875*, 2025. doi: 10.48550/arXiv.2505.18875. URL <https://arxiv.org/abs/2505.18875>.
- [13] Aleph Alpha. Alpha-moe: A fused mixture of experts megakernel. <https://github.com/Aleph-Alpha/Alpha-MoE>, 2025. Software for fused Mixture of Experts kernels compatible with vLLM and SGLang.
- [14] NVIDIA. TensorRT-LLM. GitHub, . URL <https://github.com/NVIDIA/TensorRT-LLM>.
- [15] NVIDIA. CUTLASS: CUDA templates for linear algebra subroutines. GitHub, . URL <https://github.com/NVIDIA/cutlass>.
- [16] Chenggang Zhao, Zhean Xu, Liang Zhao, Jiashi Li, Chenhao Xu, Anyi Xu, Shengyu Liu, Kexing Zhou, and Kuai Yu. Deepgemm: clean and efficient blas kernel library on gpu. <https://github.com/deepseek-ai/DeepGEMM>, 2025.
- [17] Ted Zadori, Markus Hoehnerbach, Jay Shah, Timmy Liu, Vijay Thakkar, and Tri Dao. Flashattention-4: Algorithm and kernel pipelining co-design for asymmetric hardware scaling, 2026. URL <https://arxiv.org/abs/2603.05451>.
- [18] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. Flashinfer: Efficient and customizable attention engine for LLM inference serving. In *Eighth Conference on Machine Learning and Systems*, 2025. URL <https://openreview.net/forum?id=RXPOfAsL8F>.
- [19] Philippe Tillet, H. T. Kung, and David Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *Workshop on Machine Learning and Programming Languages (MAPL)*, 2019. URL <https://dl.acm.org/doi/10.1145/3315508.3329973>.
- [20] PyTorch Team. Helion: A high-level DSL for performant and portable ML kernels. PyTorch Blog, 2025. URL <https://pytorch.org/blog/helion/>.
- [21] Lei Wang, Yu Cheng, Yining Shi, Zhengju Tang, Zhiwen Mo, Wenhao Xie, Lingxiao Ma, Yuqing Xia, Jilong Xue, Fan Yang, and Zhi Yang. Tilelang: A composable tiled programming model for ai systems, 2025. URL <https://arxiv.org/abs/2504.17577>.
- [22] NVIDIA. cuTile Python: A parallel programming model for NVIDIA GPUs. NVIDIA Documentation, 2026. URL <https://docs.nvidia.com/cuda/cutile-python/>.
- [23] NVIDIA. CuTe DSL: Python DSL for CUTLASS. CUTLASS 4 Documentation, 2025. URL https://docs.nvidia.com/cutlass/latest/media/docs/pythonDSL/cute_dsl.html.
- [24] Triton Contributors. Gluon: A lower-level GPU programming language on the Triton compiler stack. Triton Documentation and Tutorials, 2026. URL <https://github.com/triton-lang/triton/tree/main/python/tutorials/gluon>. Accessed: 2026-07-27.
- [25] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Q. Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. TVM: an automated end-to-end optimizing compiler for deep learning. In Andrea C. Arpaci-Dusseau and Geoff Voelker, editors, *13th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2018, Carlsbad, CA, USA, October 8-10, 2018*, pages 578–594. USENIX Association, 2018. URL <https://www.usenix.org/conference/osdi18/presentation/chen>.
- [26] XLA Team. XLA: Optimizing compiler for machine learning. Google, 2017. URL <https://openxla.org/xla>.
- [27] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, et al. MLIR: Scaling compiler infrastructure for domain specific computation. In *IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2021. URL <https://research.google/pubs/mlir-scaling-compiler-infrastructure-for-domain-specific-computation/>.
- [28] Siyuan Feng, Bohan Hou, Hongyi Jin, Wuwei Lin, Junru Shao, Ruihang Lai, Zihao Ye, Lianmin Zheng, Cody Hao Yu, Yong Yu, and Tianqi Chen. TensorIR: An abstraction for automatic tensorized program optimization. In *International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2023. URL <https://dl.acm.org/doi/10.1145/3575693.3576933>.
- [29] Lianmin Zheng, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, Jun Yang, Danyang Zhuo, Koushik Sen, Joseph E. Gonzalez, and Ion Stoica. Ansor: Generating high-performance tensor programs for deep learning. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2020. URL <https://www.usenix.org/conference/osdi20/presentation/zheng>.

- [30] Junru Shao, Xiyu Zhou, Siyuan Feng, Bohan Hou, Ruihang Lai, Hongyi Jin, Wuwei Lin, Masahiro Masber, Cody Hao Yu, and Tianqi Chen. Tensor program optimization with probabilistic programs. *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. URL <https://dl.acm.org/doi/10.5555/3600270.3602863>.
- [31] Bastian Hagedorn, Bin Fan, Hanfeng Chen, Cris Cecka, Michael Garland, and Vinod Grover. Graphene: An IR for optimized tensor computations on GPUs. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2023. doi: 10.1145/3582016.3582018.
- [32] Bastian Hagedorn and Vinod Grover. It’s about time: Temporal abstractions for asynchronous GPU tensor computations. In *Proceedings of the 35th ACM SIGPLAN International Conference on Compiler Construction (CC)*, 2026. doi: 10.1145/3771775.3786277.
- [33] Rupanshu Soi, Rohan Yadav, Fredrik Kjolstad, Alex Aiken, Maryam Mehri Dehnavi, Michael Garland, and Michael Bauer. Optimal software pipelining and warp specialization for tensor core GPUs. *arXiv preprint arXiv:2512.18134*, 2024. URL <https://arxiv.org/abs/2512.18134>.
- [34] Hongzheng Chen, Bin Fan, Alexander Collins, Bastian Hagedorn, Evghenii Gaburov, Masahiro Masuda, Matthew Brookhart, Chris Sullivan, Jason Knight, Zhiru Zhang, and Vinod Grover. Tawa: Automatic warp specialization for modern gpus with asynchronous references, 2025. URL <https://arxiv.org/abs/2510.14719>.
- [35] Ping Guo, Chenyu Zhu, Siyuan Chen, Fei Liu, Xi Lin, Zhichao Lu, and Qingfu Zhang. Evoengineer: Mastering automated cuda kernel code evolution with large language models, 2025. URL <https://arxiv.org/abs/2510.03760>.
- [36] Terry Chen, Zhifan Ye, Bing Xu, Zihao Ye, Timmy Liu, Ali Hassani, Tianqi Chen, Andrew Kerr, Haicheng Wu, et al. AVO: Agentic variation operators for autonomous evolutionary search. *arXiv preprint arXiv:2603.24517*, 2026. URL <https://arxiv.org/abs/2603.24517>.
- [37] Shiyi Cao, Ziming Mao, Joseph E. Gonzalez, and Ion Stoica. K-Search: LLM kernel generation via co-evolving intrinsic world model. *arXiv preprint arXiv:2602.19128*, 2026. URL <https://arxiv.org/abs/2602.19128>.
- [38] Shangzhan Li, Zefan Wang, Ye He, Yuxuan Li, Qi Shi, Jianling Li, Yonggang Hu, Wanxiang Che, Xu Han, Zhiyuan Liu, et al. Autotriton: Automatic triton programming with reinforcement learning in llms. *arXiv preprint arXiv:2507.05687*, 2025.
- [39] Bernardino Romera-Paredes, Mohammadamin Barekatain, et al. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2023. URL <https://www.nature.com/articles/s41586-023-06924-6>.
- [40] Alexander Novikov, Ngan Vu, Marvin Eisenberger, et al. AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025. URL <https://arxiv.org/abs/2512.23236>.
- [41] Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. In *International Conference on Learning Representations*, volume 2025, pages 21344–21377, 2025.
- [42] Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin godel machine: Open-ended evolution of self-improving agents, 2026. URL <https://arxiv.org/abs/2505.22954>.
- [43] Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-harness: End-to-end optimization of model harnesses, 2026. URL <https://arxiv.org/abs/2603.28052>.
- [44] Thomas Anderson, Ratul Mahajan, Simon Peter, and Luke Zettlemoyer. Self-defining systems. Technical report, Paul G. Allen School of Computer Science & Engineering, University of Washington, 2025. URL <https://foci.uw.edu/papers/whitepaper2025-sds.pdf>.